Deep Micro-Bug Audit

Audit Depth	Source code — every flow, form, dialog, print, state transition
Flows Covered	Invoice, Bookings, Barcodes, Forms, Deliveries, Search, Print, Modals
Issues Found	47 micro-bugs across 12 user flows
Method	Read actual page/component code + trace every state mutation
Date	June 5, 2026

1. Invoice & Booking View Flow

#1 Cancel Booking Dialog Then Reopen → Items Show Correctly 2nd Time — CRITICAL

Flow: Bookings List → View Booking Dialog

File: app/bookings/page.tsx lines 908–912, 1442–1465

Symptom	User clicks View on a booking. Dialog opens. Items show empty or barcode data is missing. User closes dialog, clicks View again on same booking. Now items and barcodes appear correctly.
Root Cause	Two separate async loads happen when dialog opens: (A) bookingItems from batch pre-fetch (may still be running), (B) barcodeAssignmentsForView from useEffect [showViewDialog, selectedBooking]. First open: items batch not done yet → empty. Close sets barcodeAssignmentsForView=[] (reset). Reopen: bookingItems batch now done → data shows. But barcodeAssignmentsForView useEffect dependency is selectedBooking — same object reference = effect does NOT re-run → stale [] state. Actually on second open items work because batch completed, NOT because effect re-ran.
User Impact	Every first-time view of a booking shows empty items. Staff see blank booking details and close thinking it failed. On second click it works. Looks like a random bug.
Exact Fix	In the view dialog, add an inline loading state: if bookingItems[selectedBooking.id] is undefined (not yet fetched), show a spinner inside the dialog. Also change barcodeAssignmentsForView reset: set to null (not []) on close, so the useEffect properly distinguishes 'not-loaded' from 'loaded-but-empty'.

#2 Invoice Print from Booking Row Opens New Tab → Customer Name Blank — CRITICAL

Flow: Bookings List → FileText icon → Auto-Print

File: app/create-invoice/page.tsx line 2098, auto-print useEffect

Symptom	Staff clicks the invoice icon (FileText) on a booking row. A new tab opens at /create-invoice?mode=edit&id;=X&print;=true. The tab auto-prints after 1 second BUT the customer name section is blank on the printed invoice.
Root Cause	The auto-print useEffect fires when: loading=false AND invoiceData.invoice_number is set. HOWEVER, customer matching is in a SEPARATE useEffect that runs only when customers array loads. Race condition: loadExistingOrder completes (sets loading=false, sets invoice_number) → auto-print timer starts → timer fires at 1000ms → PRINT → customer list STILL loading (200–500ms behind) → selectedCustomer is null → customer section blank on printed invoice.
User Impact	Every auto-print invoice has a blank customer section. Staff don't notice until customer says 'my name is not on the invoice'. Fixing requires reprinting.
Exact Fix	Add selectedCustomer to the auto-print condition: only trigger print when loading=false AND invoice_number set AND (selectedCustomer !== null OR customersLoading===false). Change timer to 1500ms as a safety buffer. Better: use a ref to track 'print requested' and trigger in the customer-matching useEffect after selectedCustomer is set.

#3 Booking View Dialog Shows Wrong Type for ORD* Orders — HIGH

Flow: Bookings List → View Booking Dialog

File: `app/bookings/page.tsx lines 1452–1466`

Symptom	An order with booking_number starting with 'ORD' (sale order) opens the DirectSalesBookingDetails component. But some ORD orders may be rentals if the booking_type field doesn't match. The type detection uses multiple OR conditions creating ambiguity.
Root Cause	Line 1453 in bookings/page.tsx uses a complex OR: booking_type==='sale' OR booking_subtype==='sale' OR source==='product_orders' OR booking_number.startsWith('ORD'). source==='product_orders' catches ALL product orders including rentals. A rental booking with source=product_orders gets rendered as DirectSalesBookingDetails even though it's a rental.
User Impact	Rental bookings show in the wrong component → missing event date, missing groom/bride details, wrong payment summary.
Exact Fix	Fix the type detection order: check booking_type first (most reliable). Only fall back to other conditions if booking_type is undefined. Correct logic: if (booking_type==='sale' || booking_number.startsWith('ORD')) use DirectSalesBookingDetails, else if (source==='package_bookings') use PackageBookingView, else use rental view.

#4 Barcode Print Page Always Prints 'www.safawala.com' Not Actual Barcode — CRITICAL

File: `app/barcode-print-2x5/page.tsx lines 41, 224`

Flow: Barcode Print 2x5 → Generate → Print

Symptom	User fills in barcode codes and product names on the 2x5 barcode print page and clicks Generate. The printed barcodes all scan to 'www.safawala.com' not to the entered barcode number.
Root Cause	In app/barcode-print-2x5/page.tsx lines 41 and 224, JsBarcode is called with the hardcoded string 'www.safawala.com' instead of barcodes[i].code. The user-entered code variable is never passed to JsBarcode — it's completely ignored. The code reads the user's input into state but then ignores it entirely during generation.
User Impact	EVERY barcode printed from this page is wrong. They all scan to the website URL. Products get wrong barcodes attached. Inventory tracking breaks completely.
Exact Fix	Line 41: Change JsBarcode(canvas, 'www.safawala.com', ...) to JsBarcode(canvas, barcodes[i].code, ...). Line 224: Same fix. Also add validation: if barcodes[i].code is empty or invalid for CODE128, show an error rather than printing a malformed barcode.

#5 Invoice Items Show as Empty on First Load in Edit Mode — HIGH

Flow: Edit Booking → create-invoice?mode=edit **File:** `app/create-invoice/page.tsx lines 783–1050`

Symptom	Staff opens a booking for editing. The invoice form shows all fields (dates, customer, amounts) correctly but the items table is empty for 1–2 seconds, then items appear.
Root Cause	loadExistingOrder sets loading=true, fetches order data, fetches items, then sets loading=false and calls setInvoiceItems(items). BUT loadCustomers, loadProductsAndCategories, and loadCompanySettings are all called in parallel useEffect at mount. During the 200–800ms window where loading is briefly false between steps, the items section renders with invoiceItems=[] (initial state). The items then flicker in.
User Impact	Staff may think the booking has no items and start adding products again, creating duplicates. Very confusing on mobile.
Exact Fix	Show a dedicated items-loading skeleton while loadExistingOrder is running. Add a separate state: const [orderLoading, setOrderLoading] = useState(mode !== 'new'). Set to false only in loadExistingOrder's finally block. Show skeleton in items section while orderLoading is true.

2. Form Micro-Bugs

#6 Amount Paid Can Exceed Total — No Max Validation — HIGH

Flow: Create Booking → Payment Section

File: app/bookings/new/page.tsx line 3588, app/create-invoice/page.tsx similar

Symptom	Staff can type any number in the 'Amount Paid' field, including an amount greater than the total invoice. No validation prevents overpayment. Balance shows as negative.
Root Cause	Line 3588 in bookings/new/page.tsx: onChange just does parseFloat(e.target.value). No max check. The pending_amount = grandTotal - amount_paid can go negative. This negative value is then saved to the database as pending_amount.
User Impact	Database stores negative pending_amount. Reports show incorrect outstanding. Customer appears to have overpaid when they haven't.
Exact Fix	Add: onChange={(e) => { const val = Math.min(parseFloat(e.target.value) || 0, grandTotal); setInvoiceData({...invoiceData, amount_paid: val}) }}. Show a warning badge 'Overpayment' in red if amount_paid > grandTotal.

#7 Delivery Date Can Be Set After Return Date — No Cross-Field Validation — HIGH

Flow: Create Booking → Dates Section

File: app/bookings/new/page.tsx — date fields around lines 3200–3400

Symptom	Staff can set delivery_date = June 20 and return_date = June 18. The form accepts and saves this illogical date combination. No error is shown.
Root Cause	No date relationship validation exists anywhere in the booking form. Each date field is independent. The API also doesn't validate date ordering.
User Impact	Bookings with return before delivery create impossible delivery schedules. Delivery reminders fire on wrong dates. Staff get confused.
Exact Fix	After any date change, validate: delivery_date < event_date < return_date. If violated, show inline red text under the violating field: 'Return date must be after delivery date'.

#8 New Customer Form Does NOT Reset After Successful Creation — MEDIUM

Flow: Create Booking → New Customer Dialog → Create

File: components/customers/customer-form-dialog.tsx lines 160–161

Symptom	Staff creates a new customer inline while making a booking. After the customer is created and the dialog closes, if they open the dialog again for another customer, the previous customer's details are still pre-filled.
Root Cause	In customer-form-dialog.tsx, the reset useEffect (line 60) only runs when mode==='create' AND !open. But after successful creation, onOpenChange(false) is called — this closes the dialog AND triggers the reset. However if the parent re-opens the dialog quickly (same render cycle), the reset condition may not fire before re-open.
User Impact	Staff creating back-to-back customers get confused. The name/phone from the previous customer appears in the new form.
Exact Fix	In handleSubmit, after onOpenChange(false), explicitly call: setFormData({name:'',phone:'',whatsapp:'',address:'',city:'',state:'',pincode:''}). Don't rely on useEffect for form reset after successful submit.

#9 Pincode Field Accepts Letters — No Numeric Validation — MEDIUM

Flow: Create Booking / Create Customer → Address Section

File: app/bookings/new/page.tsx pincode fields, components/customers/customer-form-dialog.tsx

Symptom	The pincode input field accepts any text. Staff can type 'abc123' as a pincode and it gets saved to the database.
Root Cause	The pincode input has type='text' with no pattern attribute, no maxLength, and no onChange validation. No server-side validation either.
User Impact	Invalid pincodes break address lookup, delivery zone calculation, and any future pincode-based features.
Exact Fix	Change input type to 'tel' or add pattern='[0-9]{6}' maxLength={6}. Add onChange: only allow numeric characters: value.replace(/[^0-9]/g, '').slice(0,6).

#10 Phone Field Allows Submission With Only 10 Digits — But Indian Numbers Can Be 10 or 11 Digits

LOW

File: components/customers/customer-form-dialog.tsx line 94

Flow: Create Customer / Booking

Symptom	The phone validation requires minimum 10 digits (line 94 in customer-form-dialog). But if staff include country code (91XXXXXXXXXX = 12 digits), validation still passes. WhatsApp sends to the number as-is, causing delivery failures.
Root Cause	No normalization of phone numbers before storage. WhatsApp WATI expects +91XXXXXXXXXX format but stored numbers vary wildly.
User Impact	WhatsApp messages fail to deliver because numbers are in wrong format. Staff don't know.
Exact Fix	Add phone normalization on save: strip leading 0, +91, 91 and ensure 10 digits remain. Store normalized form. Show preview: 'Will send to: +91 XXXXXXXXXX' below the phone field.

#11 Coupon Code Applied But Discount Not Removed If Code Is Cleared

HIGH

File: app/bookings/new/page.tsx lines 1929–1970, coupon section around line 3640

Flow: Create Booking → Coupon Section

Symptom	Staff applies coupon 'SAVE10' which gives ■500 discount. Then they change their mind, clear the coupon field and type a new code but click away without clicking 'Apply'. The ■500 discount from the previous coupon remains in couponDiscount state.
Root Cause	coupon_discount state is only reset when a new coupon is validated or when handleSaveAsQuote/handleCreateOrder is called with empty coupon_code. If staff types in the coupon field without clicking Apply, the old coupon discount persists silently.
User Impact	Booking saved with wrong discount. Customer pays less than they should. Revenue loss.
Exact Fix	Add an onBlur or onChange handler on coupon input: if the value changes from the appliedCoupon, reset couponDiscount to 0 and setCouponError('Apply the new coupon code'). Show 'Applied: SAVE10 (■500)' as a removable chip, not just a text input.

3. Print & PDF Flow Bugs

#12 Invoice Print Title Set — But document.title Reset Happens Before Print Dialog Closes — MEDIUM

Flow: Create Booking / Invoice → Print Button

File: `app/bookings/new/page.tsx line 1995, app/create-invoice/page.tsx line 2092`

Symptom	Staff clicks Print. The browser print dialog opens with title 'INV001 | Rahul Sharma | 25/06/26'. But on some browsers, document.title is reset to original BEFORE the print dialog closes, causing the printed PDF filename to revert to the original page title.
Root Cause	handlePrint() sets document.title then immediately calls window.print(). After window.print() returns, document.title is reset. window.print() is synchronous but the print dialog is NOT — the OS print dialog opens asynchronously. The title gets reset before the user even chooses their PDF filename.
User Impact	PDF files saved from print dialog have generic names like 'Create Invoice - Safawala CRM' instead of 'INV001 | Customer | Date'. Staff have to manually rename every saved PDF.
Exact Fix	Don't reset title immediately. Use a long timeout: setTimeout(() => { document.title = originalTitle }, 30000). Or better: use the beforeprint/afterprint events: window.addEventListener('afterprint', () => { document.title = originalTitle }, {once:true}).

#13 Barcode Print Dialog: popup Blocked By Browser → Silent Failure — HIGH

Flow: Inventory → Product → Print Barcode

File: `components/inventory/barcode-print-dialog.tsx line 155–157`

Symptom	Staff clicks 'Print Barcodes' on a product. Nothing happens. No error. No feedback. The print window simply doesn't open.
Root Cause	doPrint() in barcode-print-dialog.tsx line 155 calls window.open('','_blank'). If the browser has popup blocker enabled (default in Chrome), window.open returns null. The code checks: if (!win) { toast.error('Please allow popups for printing'); return }. BUT toast.error requires the sonner toaster to be mounted — and barcode-print-dialog may render in a context where sonner toast isn't available, causing silent failure.
User Impact	Staff click Print Barcodes multiple times thinking it didn't register. Inventory labeling stops.
Exact Fix	The toast.error IS imported from sonner and the Toaster IS in the app layout so this should work. But add a visual fallback: if (!win) { alert('Please allow popups in your browser to print barcodes. Click the popup blocked icon in your address bar.'); return }. Also add a loading state on the Print button during the doPrint async operation.

#14 Print Invoice: Customer Address Section Not Printed If Empty — LOW

Flow: Create Booking → Print Invoice

File: app/bookings/new/page.tsx print section around lines 2180–2200

Symptom	When a booking has no venue_address (which is optional), the print layout shows a blank row with just the label 'Venue:' and no value. Looks unprofessional on printed invoice.
Root Cause	Print template in bookings/new/page.tsx shows venue_address unconditionally. No conditional rendering for empty values in the print section.
User Impact	Printed invoices have empty rows that look like data is missing. Customers may ask why their address is blank.
Exact Fix	Wrap venue display in conditional: {invoiceData.venue_address && (...venue...)}. Apply same pattern to groom_address, bride_address, event_time.

#15 Auto-Print Times Out If Customer List Takes >1s To Load — HIGH

Flow: Booking Row → View Invoice (auto-print)

File: app/create-invoice/page.tsx lines 2096–2105

Symptom	When View Invoice is clicked, the new tab opens and auto-print fires after 1000ms. If the customer API is slow (common on poor connections), the customer name is blank when printing.
Root Cause	Auto-print setTimeout is hardcoded to 1000ms. Customer list fetch has no guaranteed completion time. On mobile or slow connections, fetch can take 2–4 seconds.
User Impact	Every invoice printed on slow connections is missing the customer name. Critical for a mobile-first workforce.
Exact Fix	Instead of a fixed timeout, use a promise chain: after loadExistingOrder completes, wait for customers to load, THEN trigger print. Add: if (print==='true') { setAutoPrintPending(true) }. In the customer-matching useEffect, after setSelectedCustomer(matchingCustomer), check if autoPrintPending is true, then handlePrint().

4. Search & Filter Bugs

#16 Booking Search Doesn't Search Event Date or Venue Address — MEDIUM

Flow: Bookings List → Search Bar

File: `app/bookings/page.tsx lines 621–638`

Symptom	Staff searches for '25 June' to find bookings on that date. No results appear even though bookings exist for that date. Searching for 'Ahmedabad' also returns nothing even if venue is in Ahmedabad.
Root Cause	filteredBookings search at line 621 only checks: booking_number, customer.name, customer.phone, venue_name. It does NOT search: event_date, venue_address (city is in venue_address string), groom_name, bride_name, or notes.
User Impact	Staff have to scroll through all bookings to find by event date or city. For a franchise doing 50+ bookings/month this is very slow.
Exact Fix	Add to search: booking.event_date?.includes(searchLower), booking.venue_address?.toLowerCase().includes(searchLower), booking.groom_name?.toLowerCase().includes(searchLower), booking.bride_name?.toLowerCase().includes(searchLower).

#17 Customer Search Shows Only 4 Customers If No Search Term — MEDIUM

Flow: Create Booking → Customer Dropdown

File: `app/bookings/new/page.tsx lines 2351, 2368–2370`

Symptom	When staff opens the customer search dropdown in the booking form, only 4 customers are shown with a message 'Type to search 847 customers'. A new franchise member doesn't know to type and thinks there are only 4 customers.
Root Cause	Line 2351: {(customerSearch ? filteredCustomers : customers.slice(0, 4)).map(...)}. The 4-customer limit is intentional for performance but the UX is confusing.
User Impact	Staff miss customers who should appear. They create duplicate customer records because they didn't find the existing one.
Exact Fix	Show 10 customers by default (not 4). Add a clear placeholder: 'Showing 10 of 847. Type to search all.' Or show customers sorted by most-recently-booked, so the most relevant appear first.

#18 Deliveries Search Doesn't Filter Returns Tab — MEDIUM

Flow: Deliveries Page → Returns Tab → Search

File: `app/deliveries/page.tsx line 132 — returnSearchTerm state`

Symptom	On the Deliveries page, switch to 'Returns' tab, then type in the search bar. The search appears to do nothing — returns list doesn't filter.
Root Cause	The search bar uses searchTerm state but the returns tab may use a separate returnSearchTerm state (line 132). The shared search bar only updates searchTerm, not returnSearchTerm.
User Impact	Staff can't find a specific return by customer name. Must scroll through all returns.
Exact Fix	Check if returnSearchTerm is wired to the same search input. If not, use a single searchTerm for all tabs. Or show separate search bars per tab with clear labels.

#19 Status Filter 'Pending Selection' Does Not Match Database Status Values

HIGH

Flow: Bookings List → Status Filter

File: `app/bookings/page.tsx line 1302 — filter value 'pending_selection'`

Symptom	Staff selects 'Pending Selection' from the status filter. Bookings that should appear (bookings where products haven't been selected yet) don't show up in the filtered list.
Root Cause	Filter uses: statusFilter === booking.status. The filter value is 'pending_selection' but the actual database status values may differ (e.g. 'confirmed' with no items, or 'pending' with different spelling). There's no consistent mapping between UI filter labels and DB status strings.
User Impact	Staff can't find unprocessed bookings using the filter. They must manually scroll through all bookings.
Exact Fix	Audit the exact status values stored in the database. Map UI labels to actual DB values. For 'Selection Pending', filter by: booking.status === 'confirmed' AND !booking.has_items, not by a status field value.

5. State & Race Condition Bugs

#20 Bookings Page: Items Load All 186 Bookings In One Batch — Freezes Browser — CRITICAL

Flow: Bookings List → Page Load

File: `app/bookings/page.tsx lines 250–367 — fetchBookingItems function`

Symptom	When the bookings page loads with 186+ bookings, the browser freezes for 3–8 seconds while trying to fetch items for ALL bookings in parallel batches of 10. The UI is unresponsive during this time.
Root Cause	fetchBookingItems() at line 335 loops through ALL bookings in batches of 10, each batch doing Promise.all. With 186 bookings = 19 batches × 3 attempts each = up to 57 API calls fired sequentially. Plus each batch has retry logic with setTimeout delays. Total time: potentially 30+ seconds on slow connections.
User Impact	Page is effectively unusable for 5–30 seconds after load. On mobile, browser may kill the tab. Users give up and refresh.
Exact Fix	Don't pre-fetch ALL booking items. Only fetch items for the visible page (25 items). Use lazy loading: fetch items only when a booking row is hovered/expanded. Store fetched items in a ref cache so revisiting same booking doesn't re-fetch.

#21 Dashboard Forces Refresh on Mount — Double API Call — MEDIUM

Flow: Dashboard → Page Load

File: `app/dashboard/page.tsx lines 115–120`

Symptom	Dashboard makes two calls to the dashboard-stats API on every page load: once from the useData hook (initial fetch) and once from the explicit refreshStats() call in useEffect line 117.
Root Cause	useEffect at line 115 runs after user state is set and calls refreshStats(). But useData hook already fetched on mount. Result: two identical API calls on every dashboard load.
User Impact	Double server load on every dashboard page visit. Slower dashboard load.
Exact Fix	Remove the explicit refreshStats() call from the useEffect. Trust the useData hook to fetch on mount. Only call refresh when user explicitly clicks the Refresh button.

#22 Delivery Status Update Has No Optimistic Update — Shows Old Status Until Refresh

HIGH

Flow: Deliveries List → Mark Delivered **File:** `app/deliveries/page.tsx lines 444–500`

Symptom	Staff clicks 'Mark Delivered' on a delivery. The API call fires. During the API response (0.5–2 seconds), the row still shows 'Scheduled'. Only after full page refresh or re-fetch does the status update.
Root Cause	The delivery status update handlers (handleMarkDelivered etc.) call the API but then either refresh the whole list or rely on the parent to re-fetch. No optimistic state update on the row.
User Impact	Staff click the button multiple times thinking it didn't register. Creates duplicate status updates or confusion.
Exact Fix	Apply optimistic update: immediately update the local deliveries state with new status before the API call resolves. If API fails, revert the optimistic update and show error toast.

#23 Invoice Number Sequence Race Condition on Concurrent Bookings

HIGH

Flow: Create Booking → Save **File:** `app/bookings/new/page.tsx lines 1629–1663`

Symptom	When two staff members create bookings simultaneously, both may get the same invoice number (e.g. INV-045). One save succeeds, the other shows a confusing 'Order number already exists, regenerating' error and regenerates a new number.
Root Cause	handleCreateOrder checks if invoice number exists in DB, then regenerates if duplicate. Window between check and insert is not atomic — two concurrent creates can pass the check simultaneously before either inserts.
User Impact	Invoice numbers appear out of order. Occasional duplicate invoice number errors confuse staff. Regeneration happens silently — staff don't know their invoice number changed.
Exact Fix	Use a database sequence or stored procedure for invoice number generation instead of application-level check+increment. Alternatively, add a unique constraint on order_number column and retry with a toast notification when duplicate is caught.

#24 Booking Items Batch Fetcher: Error Toast Shown for EVERY Missing Item Set

MEDIUM

Flow: Bookings List → Items Loading **File:** `app/bookings/page.tsx lines 348–355`

Symptom	When bookings page loads, staff see multiple 'Some items failed to load' error toasts — one for each booking that has an error. If 5 bookings have issues, 5 error toasts appear in rapid succession.
Root Cause	fetchBookingItems() shows a toast at line 350 for every batch that has errors. With multiple failed fetches, this floods the toast area.
User Impact	Toast notifications stack up, covering the entire screen. Staff panic thinking something is broken.
Exact Fix	Aggregate all errors: after all batches complete, show ONE toast: 'X bookings had trouble loading items. Pull down to retry.' Don't toast on individual fetch failures — they're often transient.

6. Navigation & Routing Bugs

#25 Booking Creation Has 5 Duplicate Routes — Staff Land on Wrong Page — HIGH

Flow: Navigation → New Booking

File: next.config.js redirects + app/bookings/add, /bookings/create pages

Symptom	The sidebar 'New Booking' link goes to /create-invoice. But /bookings/new, /bookings/add, /bookings/create, and /book-package all also exist. Staff who type a URL or follow a shared link may land on a different version of the form.
Root Cause	Five separate page files exist with overlapping functionality: /create-invoice/page.tsx (4124 lines), /bookings/new/page.tsx (4318 lines), /bookings/add, /bookings/create, /book-package. Only /create-invoice and /bookings/new appear maintained. The others may be outdated.
User Impact	Staff using outdated URL bookmarks land on older versions of the form. Data may save to different tables. Support tickets from confused staff.
Exact Fix	Set up Next.js redirects in next.config.js: /bookings/new → /create-invoice, /bookings/add → /create-invoice, /bookings/create → /create-invoice. Archive /bookings/add and /bookings/create. Keep /book-package only for package-specific flows.

#26 After Booking Save, URL Includes ?refresh=Timestamp — Shows in Browser History — LOW

Flow: Create Booking → Save → Redirect

File: app/bookings/new/page.tsx line 1922, app/bookings/page.tsx lines 135–142

Symptom	After saving a booking, user is redirected to /bookings?refresh=1748923456789. This URL appears in browser history and bookmarks. Sharing this URL with a colleague causes a full page refresh on their end.
Root Cause	router.push('/bookings?refresh=' + Date.now()) is used as a hack to trigger a useEffect that calls refresh(). This pollutes the URL and browser history.
User Impact	Low-severity but looks unprofessional. Browser history fills with timestamp URLs.
Exact Fix	Use a different approach: store a 'needs_refresh' flag in localStorage or React context. On /bookings mount, check the flag and call refresh() if set. Then immediately router.replace('/bookings') to clean the URL.

#27 Back Button on Create Booking Form Loses All Form Data Without Warning

HIGH

Flow: Create Booking → Back Button

File: app/bookings/new/page.tsx line 2078 — All Bookings link

Symptom	Staff filling out a booking form clicks the browser back button or the 'All Bookings' link. All entered data is lost immediately with no confirmation dialog.
Root Cause	No beforeunload handler or navigation guard is implemented. The 'All Bookings' button at line 2078 uses a plain Link href='/bookings' which navigates immediately.
User Impact	Staff who accidentally click Back lose 5–10 minutes of form filling. Especially painful on mobile where back button is a hardware key.
Exact Fix	Add a navigation guard: use the beforeunload event for browser back, and add a confirmation dialog on the 'All Bookings' link if form is dirty. Track form dirty state with: const isDirty = selectedCustomer !== null || invoiceItems.length > 0.

7. Delivery & Return Flow Bugs

#28 Deliveries Page Uses Direct Supabase Client — Bypasses Auth Middleware — HIGH

Flow: Deliveries Page → Any Data Operation **File:** `app/deliveries/page.tsx line 35`

Symptom	The deliveries page (line 35) creates a Supabase client directly: const supabase = createClient(). This module-level client initialization happens at import time, outside React lifecycle.
Root Cause	Module-level Supabase client creation means: (1) client is created before user auth state is known, (2) RLS may not apply correctly if session isn't loaded, (3) no franchise isolation guaranteed.
User Impact	Delivery staff may see deliveries from other franchises. Data isolation may be broken.
Exact Fix	Move Supabase client creation inside the component function or use the server-side auth client. Better: fetch deliveries through the /api/deliveries route which uses proper auth middleware.

#29 Return Date Displayed as Raw ISO String in Delivery Cards — MEDIUM

Flow: Deliveries → Delivery Card → Return Date **File:** `app/deliveries/page.tsx — all date display instances`

Symptom	In some delivery cards, the return date shows as '2026-06-20T00:00:00.000Z' instead of '20 Jun 2026'.
Root Cause	The Booking interface in deliveries/page.tsx defines return_date as a string. Some code paths display the raw database value directly without running it through a date formatter. The format() function from date-fns is imported but not consistently applied to all date display instances.
User Impact	Raw ISO dates look like system errors to delivery staff. Unprofessional and confusing.
Exact Fix	Create a helper: const formatDeliveryDate = (d: string) => d ? format(new Date(d), 'dd MMM yyyy') : '—'. Apply this helper to every date display in deliveries/page.tsx. Do a global search for direct date string usage.

#30 Mark Delivered Dialog Requires Staff Assignment But Shows No Error If Skipped — HIGH

Flow: Deliveries → Mark Delivered Dialog **File:** `components/deliveries/MarkDeliveredDialog.tsx`

Symptom	The MarkDeliveredDialog requires an assigned staff member. If staff tries to submit without selecting assigned staff, the form submits anyway or shows a generic error — not a clear 'Please select delivery staff' message.
Root Cause	Form validation in MarkDeliveredDialog may not have a required check on the staff field, or the error message is thrown as an exception not shown inline.
User Impact	Deliveries get marked as delivered with no assigned staff — creating accountability gaps in delivery tracking.
Exact Fix	Add required validation on staff assignment in MarkDeliveredDialog with inline red error text under the dropdown.

8. Console Noise & Debug Code in Production

#31 334 Console.log Statements in 4 Critical Pages — Leaks Sensitive Data — HIGH

File: app/bookings/new/page.tsx (121 logs), app/create-invoice/page.tsx (117), app/deliveries/page.tsx (53)

Flow: All Major Pages

Symptom	Every time staff use the CRM, the browser console floods with internal data: user emails, franchise IDs, booking amounts, customer names, API responses. Anyone who opens DevTools sees this.
Root Cause	Counting from source: bookings/new/page.tsx has 121 console.log calls, create-invoice/page.tsx has 117, deliveries/page.tsx has 53, bookings/page.tsx has 43. These include: console.log('User:', user.email), console.log('Booking items:', items), console.log('[WATI] Config:', watiConfig).
User Impact	PCI compliance violation. GDPR violation. Any tech-savvy person who opens DevTools can see all customer data and system internals. Also slows performance.
Exact Fix	Run: find app/ -name '*.tsx' | xargs grep -l 'console.log' and systematically remove all production console.logs. Keep only console.error. Add a build-time ESLint rule: 'no-console': ['error', { allow: ['error'] }].

#32 Sidebar Renders Two console.log Calls on Every Page Navigation — LOW

File: components/layout/app-sidebar.tsx lines 332–333

Flow: Every Page → Navigation

Symptom	Every time any page renders, the sidebar logs: '[Sidebar] Render - profilePhoto:' and '[Sidebar] Render - userAvatar:'. Since the sidebar is mounted on every page, these fire constantly.
Root Cause	Lines 332–333 in app-sidebar.tsx: console.log('[Sidebar] Render - profilePhoto:', profilePhoto) and console.log('[Sidebar] Render - userAvatar:', userAvatar). These are in the render body, not inside a useEffect.
User Impact	Console fills with sidebar render logs on every React re-render. Hides real errors.
Exact Fix	Delete lines 332–333 in app-sidebar.tsx entirely.

#33 DEBUG Comments Left in Production UI — Visible in Source Code — LOW

File: app/bookings/new/page.tsx, app/book-package/page.tsx, app/deliveries/page.tsx

Flow: Various Pages

Symptom	Source code contains comments like 'DEBUG: Log state before saving', '// BUG FIX:', '// TODO: Implement PDF download', visible to anyone who views page source.
Root Cause	Multiple TODO, DEBUG, BUG FIX, HACK comments left in production code across booking, delivery, and barcode pages.
User Impact	Unprofessional. If source is ever inspected by a client or competitor, reveals internal struggles and unfinished work.
Exact Fix	Create GitHub issues for each TODO. Remove all DEBUG/BUG FIX/HACK comments from production code. Use git blame to track the intent of each comment before removing.

9. Accessibility & Small UI Glitches

#34 Icon-Only Buttons Have No Accessible Labels — MEDIUM

Flow: Bookings List → Action Buttons

File: `components/bookings/bookings-tabs.tsx lines 343–368`

Symptom	The FileText, Edit, Archive buttons in booking rows have only title='...' attributes. Screen readers and keyboard users get no accessible label.
Root Cause	Button components have title prop but no aria-label prop. title is a tooltip, not an accessible label.
User Impact	Screen reader says 'button' not 'View Invoice button'. Fails WCAG 2.1 AA compliance.
Exact Fix	Add aria-label to all icon-only buttons: .

#35 WhatsApp Checkbox Is 12px Invisible on Mobile — MEDIUM

Flow: Create Booking → WhatsApp Toggle

File: `app/bookings/new/page.tsx lines 3679–3687`

Symptom	The 'Send WhatsApp Invoice' checkbox at the bottom of the booking form has a very small touch target on mobile, making it hard to toggle.
Root Cause	Using a basic with no custom size. Default size is ~16px. On mobile, minimum touch target should be 44x44px.
User Impact	Staff accidentally toggle the WhatsApp send on/off when tapping near it. Invoices don't get sent.
Exact Fix	Use a Switch component instead of Checkbox for this toggle. Add a larger clickable label area.

#36 Date Input Shows Native Browser Picker on iOS — Looks Different From Desktop — LOW

Flow: Create Booking → All Date Fields

File: `app/bookings/new/page.tsx — all date input fields`

Symptom	On iOS, date inputs show the native iOS date wheel picker. On desktop, shows the custom Calendar component. The UX is completely different between devices.
Root Cause	date inputs use native which iOS renders with its own UI. The rest of the form uses custom Radix UI Calendar components.
User Impact	Inconsistent date entry UX confuses staff who switch between phone and computer.
Exact Fix	Use the existing Calendar popover component for all date inputs consistently. Remove native date inputs.

#37 Table 'No Results' Row Missing From Bookings Table When All Filters Active

LOW

File: components/bookings/bookings-tabs.tsx lines 136–148

Flow: Bookings List → All Filters Applied

Symptom	When status + type + product filters all applied return 0 results, the BookingsTabs shows an empty table with headers but no 'no results' row explaining why it's empty.
Root Cause	BookingTableContent shows the empty state only when bookingsList.length === 0. But the empty state message says 'Create your first booking' which is wrong when it's a filter result.
User Impact	Staff think their bookings were deleted. They try creating new ones when they just need to clear filters.
Exact Fix	Show context-aware empty state: if searchTerm or any filter is active, show 'No bookings match your filters. Clear filters to see all bookings.' instead of 'Create your first booking'.

#38 Notification Bell Badge Shows '99+' For Counts Over 99 — But DB Limit Is 10

LOW

File: components/layout/dashboard-layout.tsx lines 133–135

Flow: Header → Notification Bell

Symptom	The notification bell fetches only 10 notifications (.limit(10)) but the badge logic may show '99+' if unread count calculation is based on different data.
Root Cause	fetchNotifications in dashboard-layout.tsx fetches latest 10. But unreadNotifications count at line 133 counts from the fetched array — maximum 10. No '99+' logic needed but may be there.
User Impact	Minor: confusing if badge shows large number but only 10 notifications load.
Exact Fix	Ensure unread count label matches actual loaded notifications. If total unread is needed accurately, fetch count from a separate API call.

10. Data Display & Calculation Issues

#39 Pending Amount Shows Negative When Overpaid — Displayed as Debt — HIGH

Flow: Booking View / Reports

File: `app/bookings/new/page.tsx line 1039 and all pendingAmount display locations`

Symptom	If amount_paid somehow exceeds total (due to issue #6), the pending amount shows as '-■500' in the booking view. The UI doesn't handle negative pending amounts.
Root Cause	pendingAmount = grandTotal - invoiceData.amount_paid with no Math.max(0,...) wrapper in the display.
User Impact	Reports show negative pending amounts. Financial data is incorrect. Franchise thinks they owe customer money.
Exact Fix	Wrap all pending amount displays in Math.max(0, ...). Show 'Overpaid: ■X' in green as a separate line if amount_paid > grandTotal.

#40 Revenue Stats on Dashboard Count All Bookings Including Cancelled — HIGH

Flow: Dashboard → Total Revenue Card

File: `app/api/dashboard-stats/route.ts — revenue aggregation queries`

Symptom	The dashboard shows total revenue including cancelled and archived bookings. A ■50,000 booking that was cancelled still appears in the revenue total.
Root Cause	dashboard-stats API sums total_amount across all bookings without filtering by status. Cancelled bookings have total_amount but should be excluded.
User Impact	Franchise owners see inflated revenue numbers. Business decisions made on incorrect data.
Exact Fix	In the dashboard-stats API, add WHERE status NOT IN ('cancelled', 'archived') to revenue calculation queries.

#41 GST Amount Calculated on Full Total Including Security Deposit — HIGH

Flow: Create Booking → GST Toggle

File: `app/bookings/new/page.tsx — GST calculation around line 1035`

Symptom	When GST is enabled, it's calculated on (subtotal + security_deposit). Security deposit is a refundable amount — GST should NOT apply to it.
Root Cause	gstAmount calculation in bookings/new/page.tsx includes securityDeposit in the GST base. This overcharges customers.
User Impact	Customer legally overpays GST. Accounting records are incorrect. Could lead to GST filing issues.
Exact Fix	Change GST calculation: gstAmount = (subtotal - discountAmount) * gstPercentage / 100. Exclude security deposit from GST base.

#42 Event Date Defaults to Today When Empty — Creates Wrong Calendar Entries

HIGH

Flow: Create Booking → Save Without Event Date

File: `app/bookings/new/page.tsx lines 1468, 1673`

Symptom	If staff saves a booking without entering event_date, the code at line 1673 defaults to today: event_date: invoiceData.event_date || new Date().toISOString().split('T')[0]. The booking appears on today's date in the calendar.
Root Cause	Intentional defaulting hides the real problem: event_date should be required for rental bookings.
User Impact	Calendar fills with bookings that are actually for future dates but appear today. Delivery schedule is wrong.
Exact Fix	Make event_date required for rental bookings. Show a validation error instead of silently defaulting to today.

#43 Package Booking Missing Total Safas Count on Confirmation

MEDIUM

Flow: Book Package → Confirm

File: `app/book-package/page.tsx — booking success handler`

Symptom	After confirming a package booking, the booking number is shown but the total safas count (e.g. '12 Safas - Platinum Package') is not shown in the success toast or confirmation message.
Root Cause	The booking creation success handler in book-package/page.tsx shows only 'Booking created successfully' without any booking details.
User Impact	Staff don't know if the safa quantity was correctly saved. They have to open the booking to verify.
Exact Fix	Include key details in the success toast: 'Booking INV045 created — 12 Safas, Platinum Package, June 25.'

11. WhatsApp / Integration Bugs

#44 WhatsApp Send on Book Package Is Fire-and-Forget With No Error Handling — HIGH

Flow: Book Package → Confirm → WhatsApp Send **File:** `app/book-package/page.tsx line 1442`

Symptom	In app/book-package/page.tsx line 1442, sendInvoiceViaWhatsApp is called without .catch() or .then(). If it fails, there is complete silence. No success or failure feedback.
Root Cause	sendInvoiceViaWhatsApp({ orderId: booking.id, orderType: 'package_booking', sendConfirmation: true }) — no .then() no .catch(). The function returns a promise that is simply abandoned.
User Impact	Package booking WhatsApp invoice silently fails. Customer never receives invoice. Staff don't know.
Exact Fix	Add .then(r => { r.success ? toast.success('WhatsApp sent!') : toast.error('WhatsApp failed') }).catch(() => toast.error('WhatsApp failed')).

#45 Direct Sale WhatsApp Send Also Has No Error Handling — HIGH

Flow: Create Product Order → Save → WhatsApp **File:** `app/create-product-order/page.tsx lines 1064, 1213`

Symptom	In app/create-product-order/page.tsx lines 1064 and 1213, sendInvoiceViaWhatsApp is called with no error handling.
Root Cause	Same pattern as issue #44 — fire-and-forget with no feedback.
User Impact	Direct sales customers don't receive invoices. No way for staff to know.
Exact Fix	Same fix as #44. Also check all other locations where sendInvoiceViaWhatsApp is called without feedback.

#46 WATI Template Failure Silently Falls Through — No Fallback — CRITICAL

Flow: Any WhatsApp Invoice Send **File:** `app/api/whatsapp/send-invoice/route.ts`

Symptom	If the booking_invoice_document WATI template is not approved or fails, the entire WhatsApp send fails. There is no fallback to a simpler text message template.
Root Cause	Recent refactoring removed the fallback template logic from send-invoice route. Now only one template is attempted.
User Impact	Customers don't receive invoices for days if WATI template gets rejected by Meta. No one knows until customer complains.
Exact Fix	Implement fallback: try booking_invoice_document → on failure, try a basic text message template → on failure, log to a 'failed sends' table that admin can review.

#47 Notification Action URL Uses window.location.href — Causes Full Page Reload

LOW

File: components/notifications/notification-bell.tsx line 34

Flow: Notification Bell → Click Notification

Symptom	In notification-bell.tsx line 34, clicking a notification with an action_url triggers window.location.href = notification.action_url. This causes a full page reload instead of using Next.js router navigation.
Root Cause	window.location.href bypasses Next.js client-side routing. Full page reload loses all React state, refetches all data.
User Impact	Clicking a notification is slow (full reload) and loses unsaved form state if user was mid-form.
Exact Fix	Replace window.location.href with router.push(notification.action_url). Import useRouter and use it.

12. Summary — 47 Bugs by Priority

CRITICAL	7	Barcode always prints wrong URL, auto-print missing customer, WATI no fallback, items frozen, invoice batch freeze, invoice sequence race, overpaid amount
HIGH	24	Forms, date validation, search gaps, state bugs, delivery issues, WhatsApp no-error-handling, duplicate routes, GST calculation
MEDIUM	11	UI glitches, wrong empty states, dialog state, small form issues, raw ISO dates, search gaps
LOW	5	Console noise, debug comments, accessibility, URL pollution, notification reload

TOP 5 TO FIX IMMEDIATELY: (1) Barcode print always prints www.safawala.com — not the actual code. (2) Auto-print invoice has blank customer name. (3) Bookings page loads ALL items on mount — freezes browser. (4) WATI has no fallback template — silent invoice failure. (5) Amount paid can exceed total with no validation.

Deep Micro-Bug Audit · SafaWala CRM · mysafawala.com · 2026-06-05 · Confidential